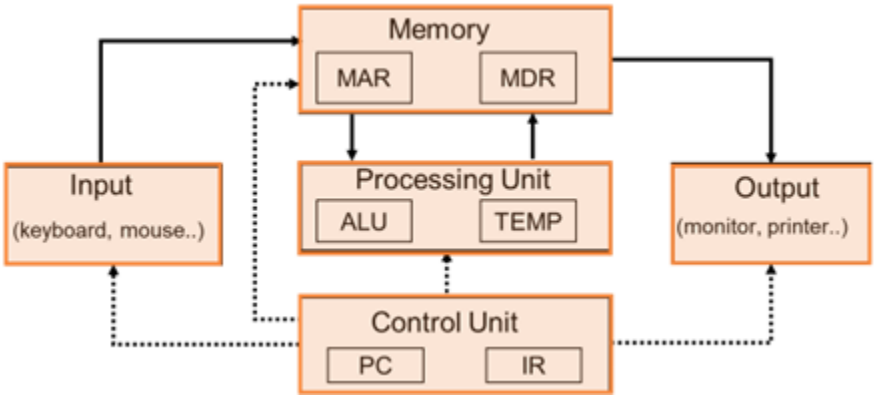
Von Neumann and Harvard Architecture

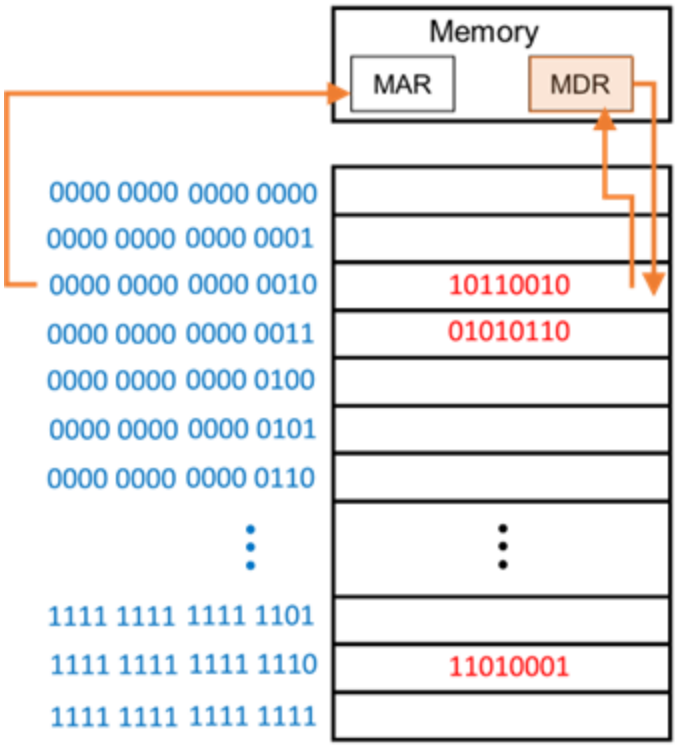
Introduction:
A computer helps us perform some tasks. What do we need to fulfill this goal?
The first need is a computer program that defines the task to the computer. A computer program is a set of instructions that help achieve the required task. The computer then performs the task by executing or simply following the instructions defined in the computer program. So, in simple terms, a computer is something that can read, interpret, and execute instructions. A fundamental model of a computer is the von Neumann model.

von Neumann Model:
In the year 1945, von Neumann proposed a computer design based on the concept of the stored program, in which program instructions and data are held in the memory.
As you can see from the diagram, the von Neumann model of a computer consists of five key parts:

1. Memory: It stores data and instructions of the computer program. The program itself requires some data to carry out the work it is defined to perform. This data is also either contained in the memory or obtained from one or more of the input devices.
2. Processing Unit: It executes instructions.
3. Control Unit: It decides the order of execution (or sequencing) of the instructions.
4. Input: The program may provide some external data from input devices like the keyboard.
5. Output: It is the result of execution. The result or the output of the execution of the program is provided on the output devices like a monitor.



von Neumann Model - Memory:
A memory unit can be constructed using gates and latches. Most computer systems have large-sized memory with large storage space. The diagram here depicts a 2^{16} by 8-bit memory.



2^{16} by 8 bits Memory

Memory can be seen as

- an array of (2^k) memory locations
- with each location having m-bit storage.

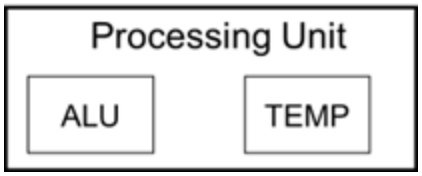
Thus, every memory location can be identified by a k-bit address and it can store m bits of content. In the diagram,

- k = 16, i.e., address space of 2^{16} uniquely identifiable locations
- m = 8, i.e., addressability of 8 bits

LC3 is a type of simple computer that has an address space of (2^{16}) locations and an addressability of 16 bits.

Let us try to understand the role of the Memory Address Register (MAR) and Memory Data Register (MDR) of the memory module. First, the address of the location to be read must be placed in the Memory Address Register (or MAR). When the read enable signal is asserted on the memory, the content pointed to by the address in the MAR will then be placed in the memory's data register (the MDR). In the above diagram, if the MAR is given, the address of location 3 (that is, 14 zeros followed by a 1 and then a 0), the content 10110010 will be read into the MDR. For a write operation work, the address of the memory location to be written into is first updated in the MAR. The content to be written in that location is to be placed in the MDR. When the write enable signal is asserted on the memory, the content of the MDR gets written in the memory location pointed to by the MAR.

von Neumann Model – Processing Unit:

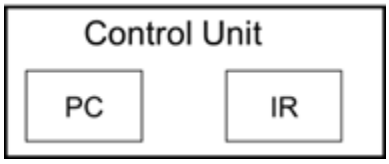


The processing unit of a computer can consist of multiple complex functional units, each performing one type of operation (e.g., a divide operation or a square root operation). In the basic von Neumann model, the simple processing unit consists of the ALU, or the Arithmetic and Logic Unit. As the name suggests, the ALU performs arithmetic operations like ADD or SUBTRACT and logical operations like AND, OR, or NOT.

Most computers will also provide a small amount of storage close to the ALU, in order to allow the results of these arithmetic and logic operations to be temporarily stored. This is useful when the result is used in subsequent instructions soon. Normally, the result of operations could be stored and read from memory. But memory access is time-consuming, much more than it takes to perform an arithmetic or logic operation. Temporary storage within the Processing Unit, in the form of registers, allows for avoiding the longer access time of the memory, improving performance. As we will later learn, the LC-3 computer has eight registers, R0 to R7, each containing 16 bits.

The data elements that the ALU processes are normally of fixed size. This is referred to as the word length of the computer. The data elements are called words. The ALU of the LC-3 computer has a word length of 16 bits and processes 16-bit words at a time. As a comparison, processing units in most modern-day computers and cell phones have either 64-bit or 32-bit word lengths.

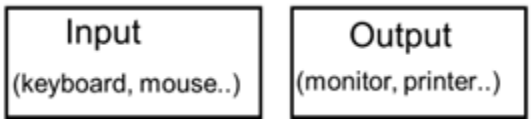
von Neumann Model – Control Unit:



The control unit **orchestrates** or decides the order of execution (or the sequencing) of the instructions in a program. In that sense, the control unit also coordinates all actions needed for the execution of an instruction.

The control unit contains two registers, which are used to track program and instruction execution. The program counter (also sometimes called the Instruction Pointer) holds the address of the next instruction to be fetched from memory. The second is the Instruction Register, which holds the instruction that is currently being executed. Note that an instruction may take many *steps* to complete.

von Neumann Model – Input and Output:



The input and output components are also called peripheral devices, as they act as accessories for the main processing function of the computer. However, they are equally important in the overall functioning of the computer. Input devices provide the information required for a program to perform its work. For example, a keyboard can act as the basic input device providing this information.

Output devices are useful to display the result of execution of the computer program. A monitor can act as the basic output device displaying the result of execution of the program.

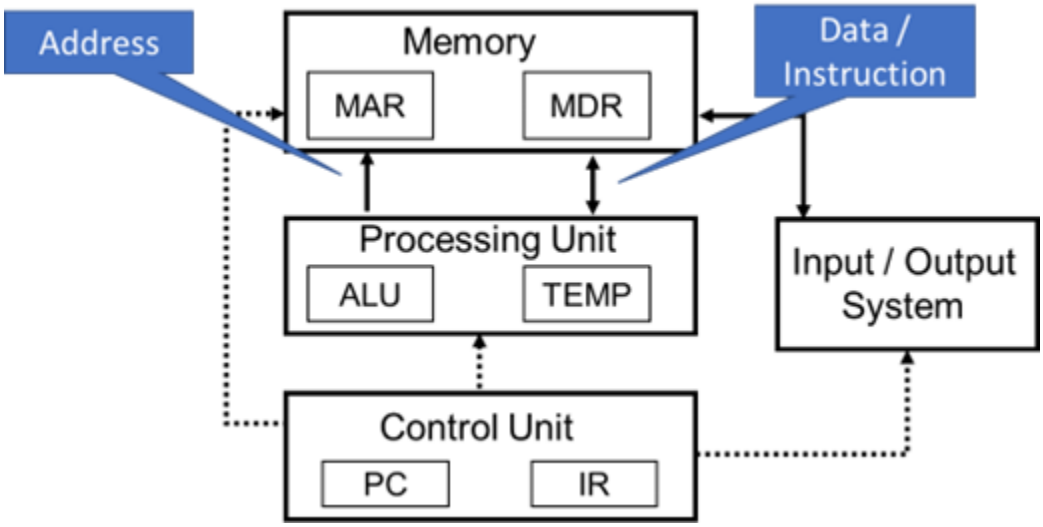
Modern-day computer systems have many other types of input and output devices, like mouse, digital scanners, LED displays, and so on. Mechanisms for communicating with these autonomous devices need to be developed. These mechanisms are part of a computer's input/output or I/O operations.

von Neumann Bottleneck

In the von Neumann model, memory is the component that stores the program instructions and the data. The program instructions and the data flow from the memory to the processing unit over the system bus. The system bus itself consists of the address bus, the data bus, and the control bus. When an address is written by the processing unit on the address bus, the contents of the memory location corresponding to that address are fetched and transferred over the data bus. This content can be either program instruction or data.

As computer programs become complex and the data needed for these programs grows, the system bus becomes a bottleneck. Due to a single memory holding both data and instructions and a single system bus, the CPU cannot simultaneously read an instruction and read or write data from or to the memory; that is, the processing unit would either fetch an instruction or data, but not both together. Due to the von Neumann bottleneck, the processer remains idle for longer periods due to these limitations.

A computer system is referred to as being "memory bound" if it does not gain performance by increasing CPU speeds.



This severely impacts the performance of a computer designed on the von Neumann model. Over the years, while the other components of the computer have become increasingly fast, the transfer rates on the system bus have only seen modest improvements. So, while processor speeds have increased significantly over the years and memory densities have grown allowing more data storage in less space, the system bus transfer rates have impaired the full utilization of these gains. No matter how fast a processor works, it will only be sitting idle if it does not get program instructions and data fast enough over the system bus. This is what we have come to know as the von Neumann bottleneck.

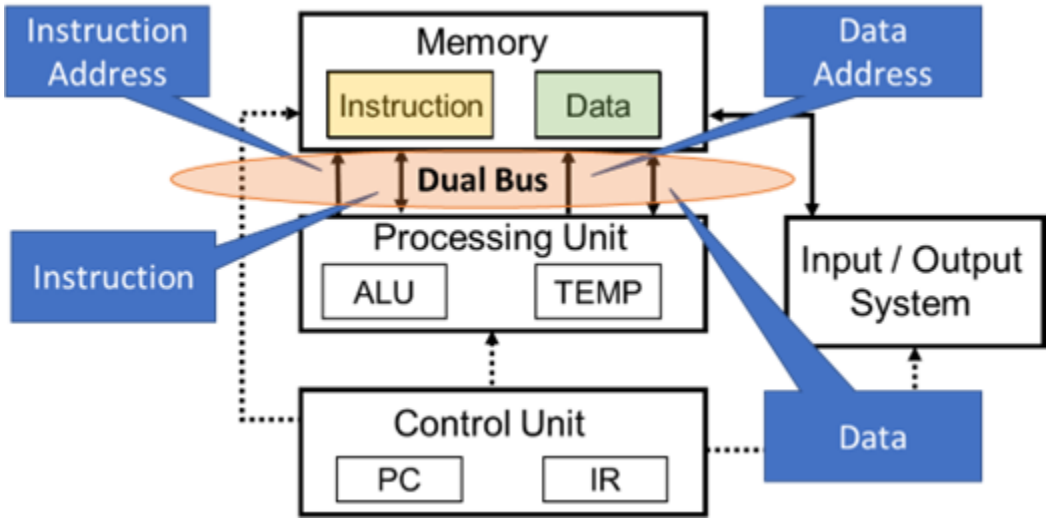
Potential Solutions to the von Neumann Bottleneck

1. Caching – Caching involves adding small storage units in the Processing Unit to help speed up the operations. These small storage units are called *Caches*. Typical cache configurations consist of L1, L2, and L3 caches, where L1, L2, and L3 are the different cache levels. L1 caches are the fastest storage units but are also expensive, leading to their sizes being kept

small. L3 caches, on the other hand, are not as fast but are less expensive, allowing larger sizes for the L3 caches. L2 caches fall somewhere between L1 and L3 caches in terms of their speed, cost, and sizes.

2. System-on-Chip (SoC) – Modern-day system-on-chips (or SoCs, as they are called in short) attempt to overcome the von Neumann bottleneck by integrating processing and storage resources on a single chip. This also eliminates much of the data transfer over the system bus, improving the overall performance.
3. Harvard Architecture – The Harvard architecture performs better in terms of the system bus transfer rates compared to von Neumann architecture due to its slightly different system design.

Harvard Architecture



Unlike the Von Neumann model, which stores and retrieves program instructions and data from memory using a single bus, the Harvard architecture keeps the data and instructions in their own separate memory spaces.
The main memory is divided into:

- instruction memory and
- data memory.

Alongside the bifurcation of the memory, the system bus is also duplicated in Harvard architecture. Two separate buses are used in the Harvard architecture, one for instruction access and the other for data. Having separate buses for instruction and data enables the CPU to access instructions and read or write data at the same time. Thus, two memory accesses can be made during any one instruction cycle, improving the overall performance of the system.
Accordingly, Harvard architecture has the following bus types:

- the Instruction Address Bus (on which the instruction address is set),
- the Instruction Bus (on which the instruction is transferred),
- the Data Address Bus (on which the data address is set), and
- the Data Bus (on which the actual data is transferred).

An outcome of the Harvard architecture is that computer systems can be designed to use different types of memory for instructions and data; that is, the two memories may not even share the same characteristics. As an example, computer systems may be built to store instructions in read-only

memory, whereas data generally gets stored in read–write memory. Another example can be of computer systems that have different sizes for the two memories, maybe much more instruction memory than data memory. This means that the instruction addresses need to be wider than the data addresses for such a system. Finally, caching can still be used with Harvard architecture to further improve performance. The cache memory can be divided into instruction cache and data cache when used as per the Harvard architecture design approach.

von Neumann vs. Harvard Architecture

- Cost: von Neumann design is cheaper due to the single bus design. However, the Harvard architecture is costlier due to its dual bus design.
- Performance: Harvard architecture performs better due to its dual bus design and therefore, scores on the performance parameter in comparison to the von Neumann design.
- Use: Due to the cost vs. performance trade-offs - von Nuemann architecture is used for personal and small computers - Harvard architecture is used for micro-controllers and signal processors (DSP)

LC-3 Architecture

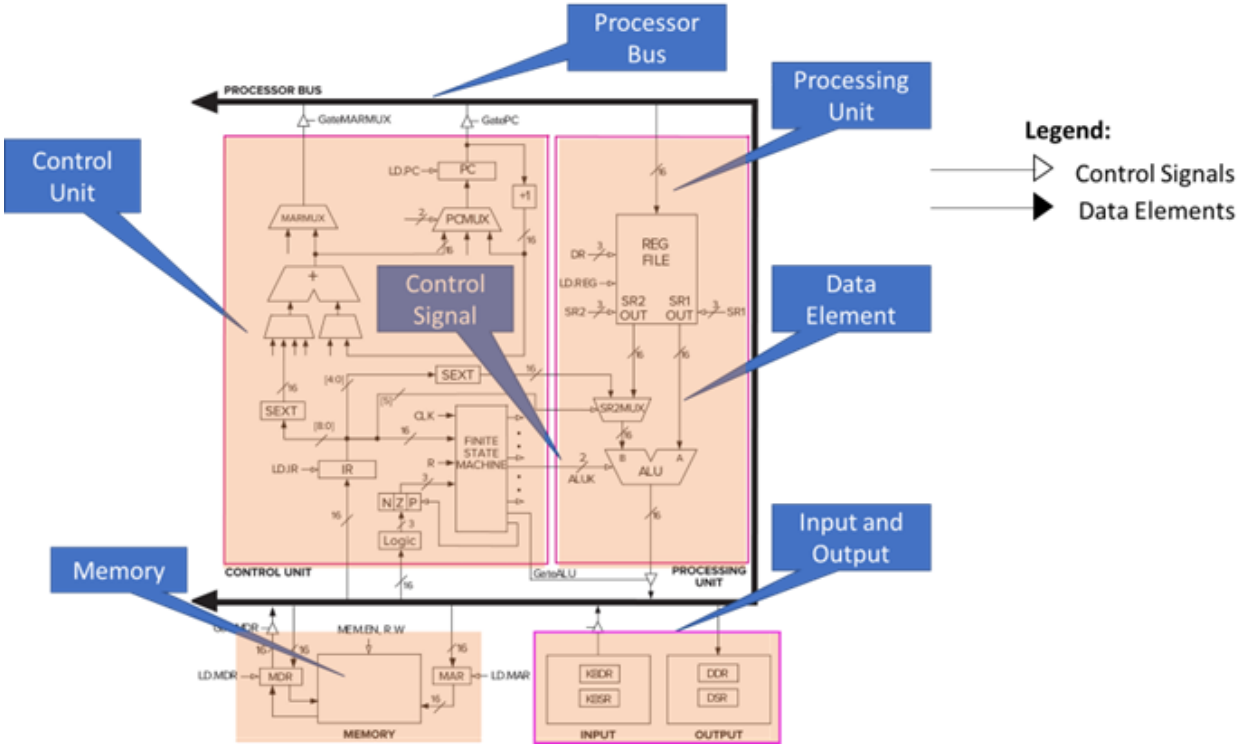


Image Source: Yale N. Patt & Sanjay J. Patel (2019). *Introduction to Computing Systems,* 3rd Edition. McGraw Hill.

In this diagram displayed here, we can see the architecture of the LC-3 computer. There are control signals and data elements flowing across the digital components. In the diagram, the control signals are the arrows with the unshaded arrowhead. As an example, the 2-bit control signal to the ALU controls the operation to be performed by the ALU. The data elements are depicted by shaded arrowheads in the diagram. Again, as an example, check the data inputs to the ALU, which are the operands for the operation to be performed in the ALU.

At a high level, you can identify the five parts of the von Neumann model in the LC-3.

- The first is the memory.

- The second and third parts are the Input and Output.
- The fourth part is the Processing Unit.
- The fifth part is the Control Unit.

The Processor Bus or the System Bus connects the different parts to transfer information to and fro from each component.

LC-3: Memory

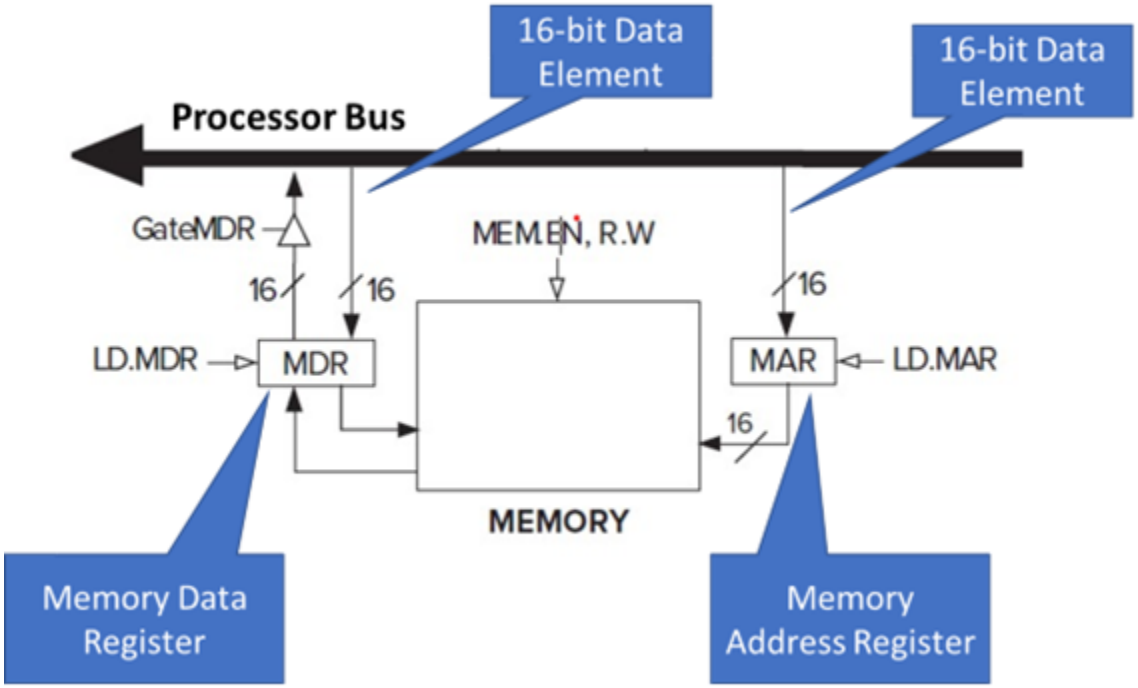


Image Source: Yale N. Patt & Sanjay J. Patel (2019). *Introduction to Computing Systems*, 3rd Edition. McGraw Hill.

In this diagram displayed here, we can see that apart from the main memory storage area, the memory unit also consists of two registers:

- the memory address register (MAR) and
- the memory data register (MDR).

The MAR contains the address of the memory location, whereas the MDR contains the data that is to be written to memory or what is to be read from the memory. As we can see from the diagram, the MAR is getting filled with a 16-bit data element,indicating that the MAR is a 16-bit register. Accordingly, we can say that the LC-3 has a memory address space of 2^{16}, or that there are 2^{16} uniquely identifiable memory locations. Similarly, we can see that the MDR is also 16-bit sized, indicating that each memory location can contain a data element of size 16 bits. We, therefore, say that the LC-3 has an addressability of 16 bits.

LC-3: Input and Output

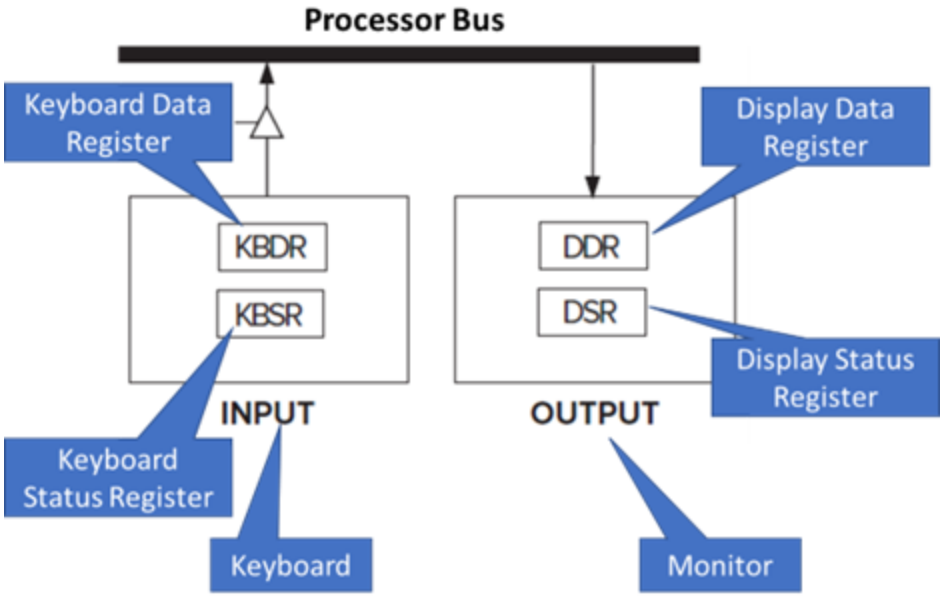


Image Source: Yale N. Patt & Sanjay J. Patel (2019). *Introduction to Computing Systems*, 3rd Edition. McGraw Hill.

While computer systems can have different types of input and output devices, in the LC-3, the input device is the Keyboard.

The LC-3 input module consists of two registers:

- the Keyboard Data Register, which holds the ASCII codes of the keys that we type on the keyboard and
- the Keyboard Status Register, which maintains some form of status information corresponding to the input device

The LC-3 output device is taken as a Monitor, and the output module also consists of two registers:

- the Display Data Register, which holds the ASCII code for the data that needs to be displayed on the monitor and
- the Display Status Register, which has the associated status information.

LC-3: Processing Unit

The main functional unit within the Processing Unit of the LC-3 is the Arithmetic and Logic Unit (or the ALU). When we studied the von Neumann model, we discussed that the ALU performs arithmetic operations like ADD or SUBTRACT and logical operations like AND, OR, or NOT. The ALU of the LC-3 implements a subset of these operations. In terms of the arithmetic operations, the LC-3 ALU only performs the ADD operation, and in terms of the logical operations, the LC-3 ALU performs the AND and the NOT operations. Since the AND and NOT operations are *logically complete*, that is, they can be used to express all possible truth tables using a Boolean expression created using them; the LC-3 can implement any bitwise logical function using just these two logical operations.

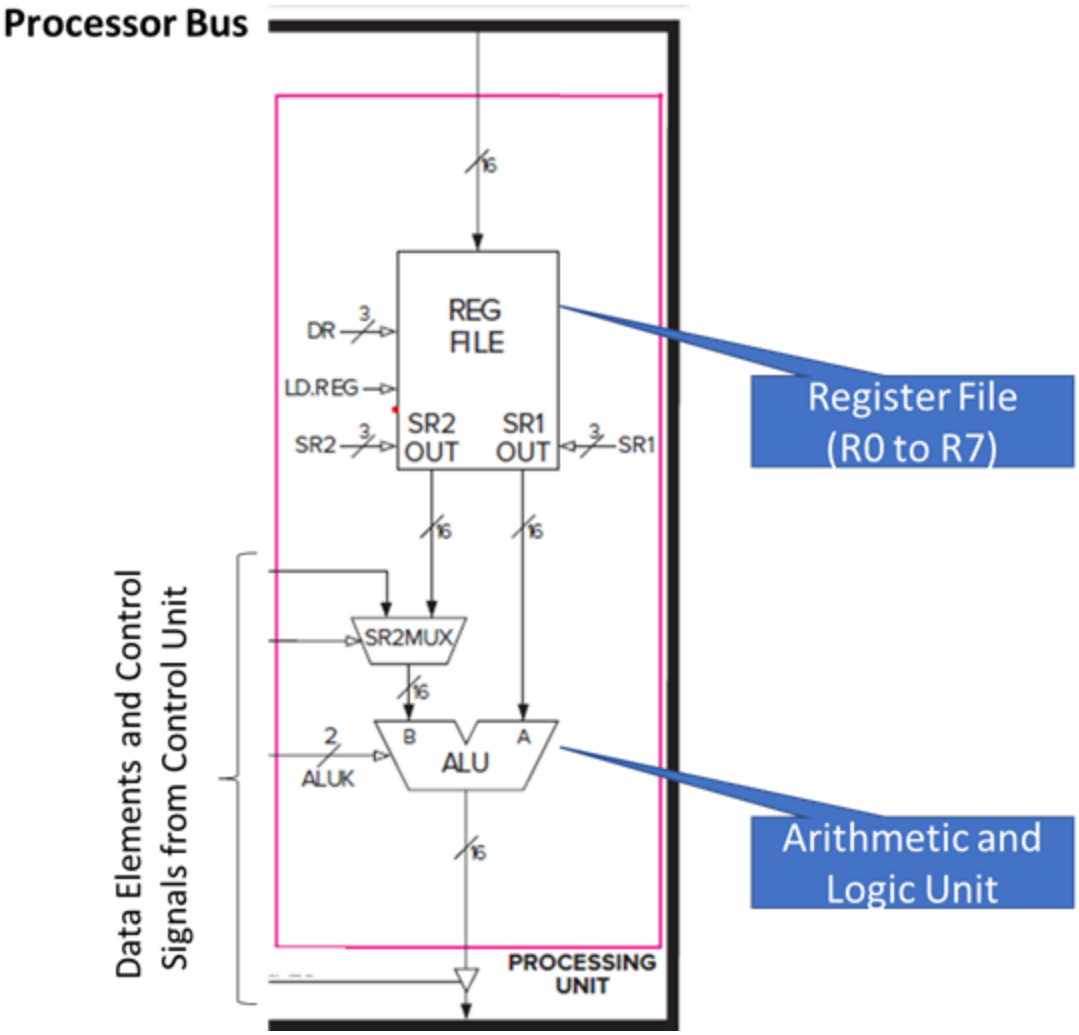


Image Source: Yale N. Patt & Sanjay J. Patel (2019). *Introduction to Computing Systems*, 3rd Edition. McGraw Hill.

In the von Neumann model, we noted the importance of having a small amount of storage close to the ALU, in order to allow the results of these arithmetic and logic operations to be temporarily stored and used in subsequent instructions. The register file depicted in our diagram is this storage, which consists of eight registers, R0 to R7, each containing 16 bits.

LC-3: Control Unit

In the LC-3 Control Unit, the main functional unit is the Finite State Machine. The finite state machine is the heart of the computer. It is the finite state machine that decides how the processing is done by the LC-3 computer. As an example, one of the outputs from the finite state machine is the 2-bit control signal to the Arithmetic and Logic Unit inside the Processing unit. This 2-bit control signal decides the operation that the ALU is supposed to perform – arithmetic ADD, or bitwise AND or NOT.

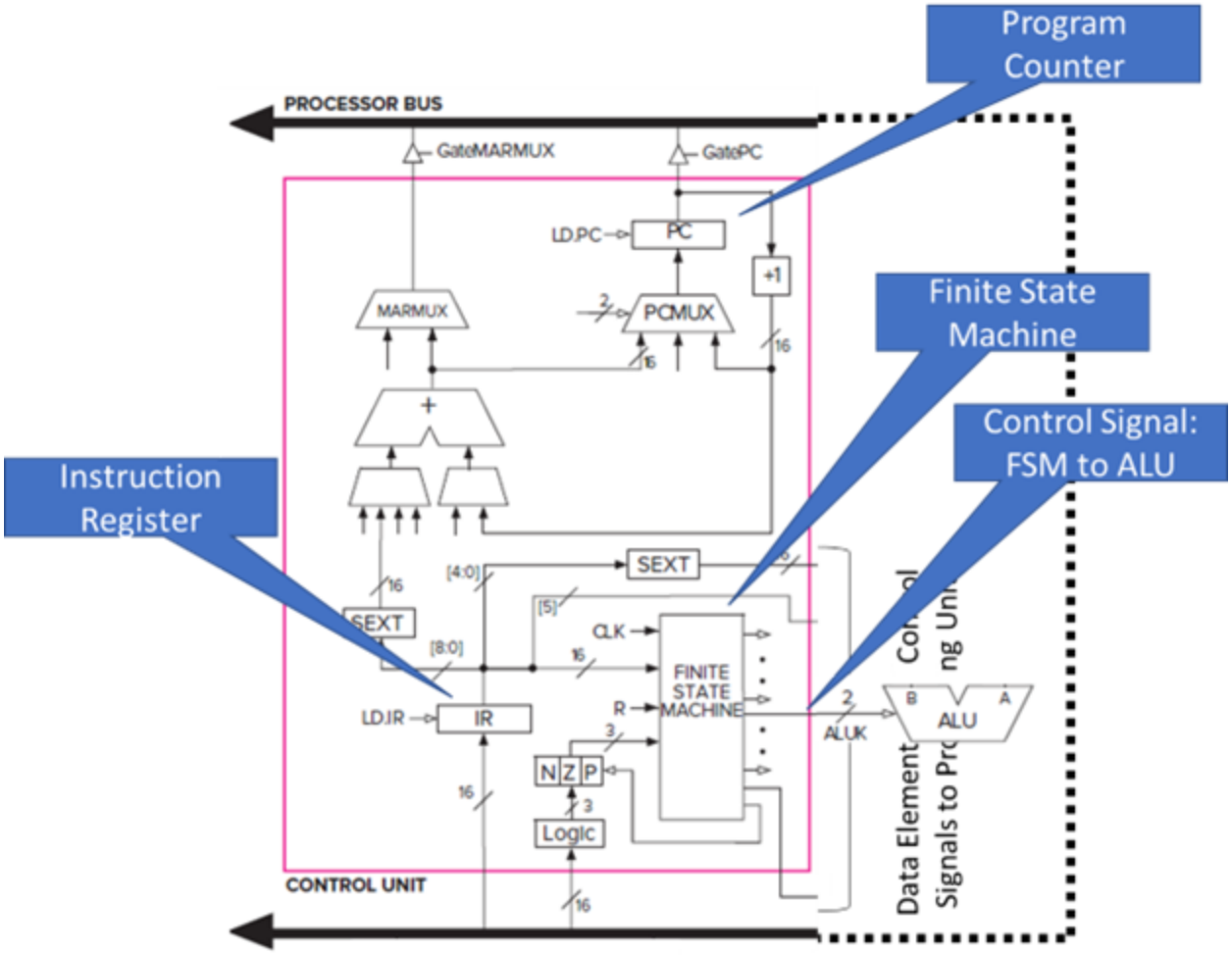


Image Source: Yale N. Patt & Sanjay J. Patel (2019). *Introduction to Computing Systems*, 3rd Edition. McGraw Hill.

In the von Neumann model, we have seen that the control unit contains two key registers. These registers are used to track the program and instruction execution. The program counter (also called the Instruction Pointer) holds the address of the next instruction that is to be executed after being fetched from memory. On the other hand, the Instruction Register holds the instruction that is currently being executed, and since this decides the activities that the computer will carry out, it is one of the key inputs to the finite state machine.

Instruction Processing

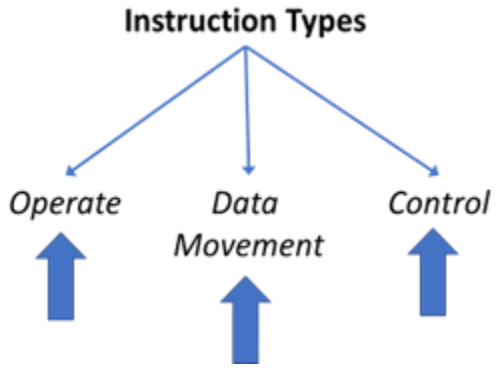
Instruction: Definition

An instruction is the basic unit of work done by the computer; therefore, the speed of a computer's processor is often measured in terms of instructions processed per second.

An instruction is composed of two parts: the opcode and the operands. The opcode defines the operation to be performed. In other words, it defines what the instruction does. On the other hand, the operand is the data or location on which this operation needs to be performed. For example, when we say 5 + 4 = 9, the + is the operation to be performed (or the opcode), and 5 and 4 are the operands on which this operation is performed. 9 is then the result of this operation.

A computer's instructions and the formats of these instructions define the computer's instruction set architecture (or ISA, in short).

Instruction: Types



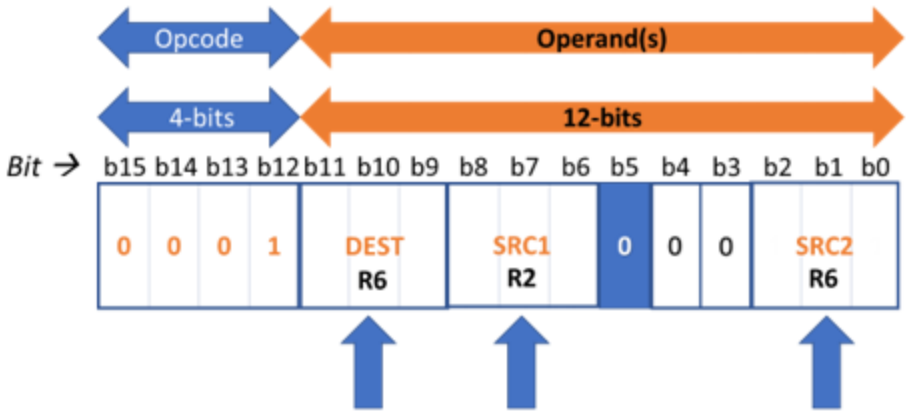
The operate instruction types are those that "operate" on data or process the data. When discussing LC-3 as an example of the von Neumann model, we saw that the ALU performs three operations: ADD, AND, and NOT. These are all instructions of the Operate instruction type.

The data movement type instructions move data between the memory or registers to and from the processing unit and the input and output devices.

Lastly, the control instruction types are used to change the otherwise sequential nature of instruction execution.

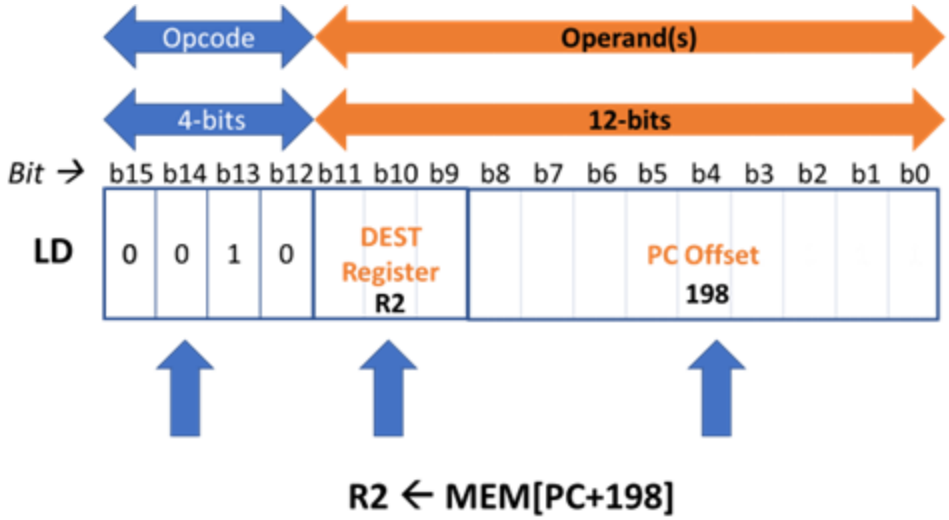
For example, instructions are usually executed sequentially, starting from instructions 1 to 2 to 3, and so on. But what if, after instruction 3, either instruction 4 or instruction 8 should be executed based on some condition? Control-type instructions allow us to change and control the sequence of execution of instructions. This allows us to develop different ways in which the execution control can flow within our program. It can have execution loops if we wish to repeat a set of instructions multiple times, or it could have branches if our execution depends on some condition. Something like, if condition A is TRUE, then do X; else, do Y.

Instruction: Example 1



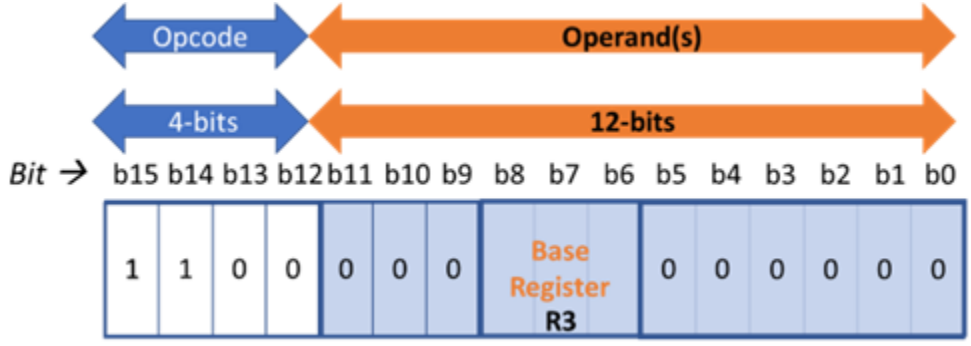
R6 ← R2 + R6

The first example we will look at is the ADD instruction, which is an **operate** type instruction from the LC-3 ISA. The diagram shows the 16 bits of the LC-3 instruction, that is, bit positions 0 to 15.
Bit positions 12 to 15, the 4 MSB bits, define the opcode. This 4-bit opcode can have 2^4 or 16 unique opcodes or 16 different instructions. In LC-3 ISA, the ADD operation has an opcode of 0001.
The operands for the ADD instruction are specified using some of the next 12 bits. An ADD instruction needs three operands, two source operands to be added, and a third “destination” operand to store the result of the addition.
An LC-3 computer contains eight registers for storing data. In our example, we will see that the ADD instruction is provided with the two source operands that use two of these eight registers. The result of the ADD operation is also put into one of these eight registers. As there are eight registers, we need three bits to identify each register.
Bits 9 to 11 describe the destination register that will store the result of the ADD operation. Let us assume that this is register R6.
The register containing the first value to add, or source 1 is indicated by bits 6 to 8. Let us assume that this source register is Register R2.
Similarly, source 2 is specified by a register identified via bits 0 to 2. Let us assume that this source register is also Register R6.
Bit 5 has a special interpretation, which we will study later. In our example, Bit 5 has a value of 0.
Bits 3 and 4 are unused in our example and are also given the value 0.
Thus, the instruction tells the computer to add the contents of register R2 to the contents of register R6 and then store back the result of this ADD operation in register R6 itself.
Instruction: Example 2



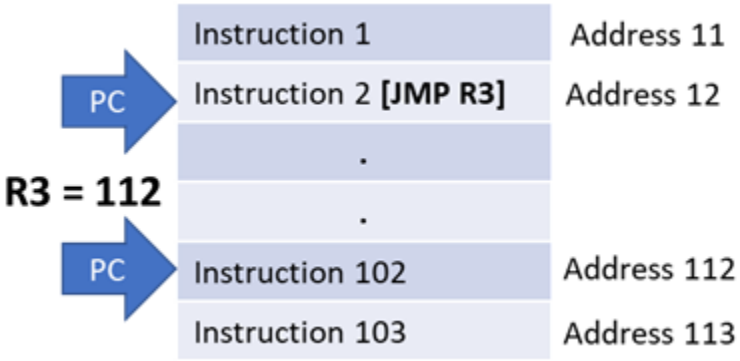
This is an example of the data movement instruction type. Load or the LD instruction from the LC-3 is used to read a value from a memory location and copy that value into a register.
The 16 bits of the LD instruction are as follows.
Bit positions 12 to 15, the 4 MSB bits, define the opcode. In LC-3 ISA, the LD operation has an opcode of 0010.
Bits 9 to 11 describe the destination register that will store the result of the LD operation. Let us assume that we will use register R2 to store the value read from memory.
The second operand must be the memory location that is to be read. We can specify the address of a memory location in different ways called ***addressing modes***. Our current example uses an addressing mode called the PC+offset addressing mode. In this mode, we use bits 0 to 8 to specify an offset value that is added to the program counter (or PC) to get the address of the memory location to be read. The bits include a 2's complement integer, that is, the value 198 in our example. So, the computer shall add 198 to the current address contained in the program counter, which provides the address of the memory location to be read. Thus, this instruction reads the value stored in the memory location pointed to by PC+offset (198) and loads it into register R2.

Instruction: Example 3



At last, let us look at an example of the control instruction type. Here, the jump or the JMP instruction from the LC-3 ISA is considered. The jump instruction is used to set the program counter with a value contained in a register, making it the address of the next instruction that will be fetched. The diagram here depicts the 16 bits of the LC-3 jump instruction.

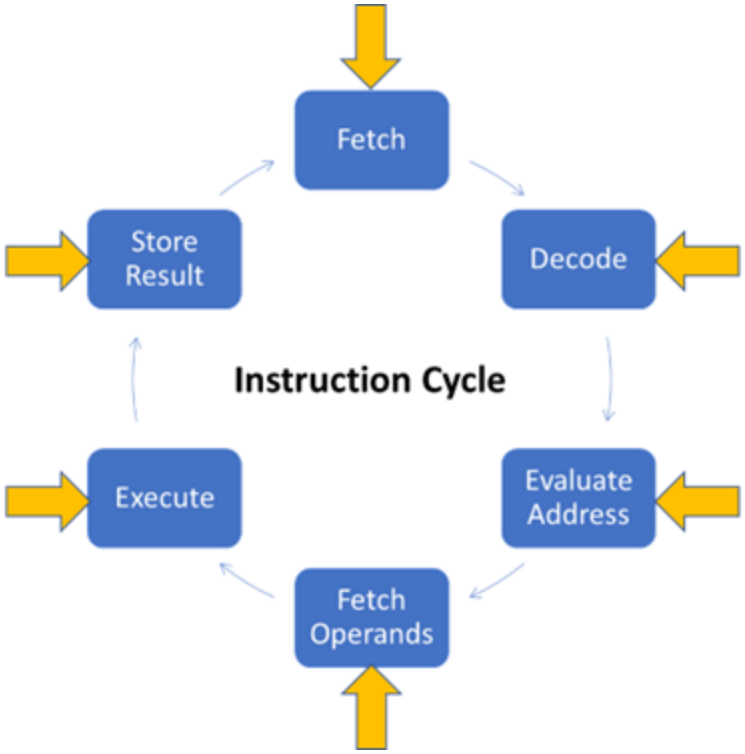
Bit positions 12 to 15, the 4 MSB bits, define the opcode. In LC-3 ISA, the JMP operation has an opcode of 1100.
Bits 6 to 8 describe the register whose value will be copied into the program counter. This is called the base register, which in this example, is register R3. Bits 9 to 11 are unused and set as 0. Also, bits 0 to 5 are also unused and set as 0. The result of execution of this example instruction is to load the PC with the value contained in the register R3, thus changing the sequence of operation of the instructions.
Look at this example instruction sequence depicted in the below diagram.



Let us assume that the program counter is at instruction 2, on address location 12, which is a jump instruction, JMP R3—assuming that the value of R3 was 112. After execution of this instruction, the PC is assigned the value of R3. Thus, this will make the program counter bypass all intermediate instructions and jump directly to address location 112, having the instruction 102.

Instruction Cycle

An instruction processing cycle or instruction cycle consists of six phases. Not all instructions require all six phases, and if an instruction does not require a particular phase, it is simply skipped, and we move on to the next phase.



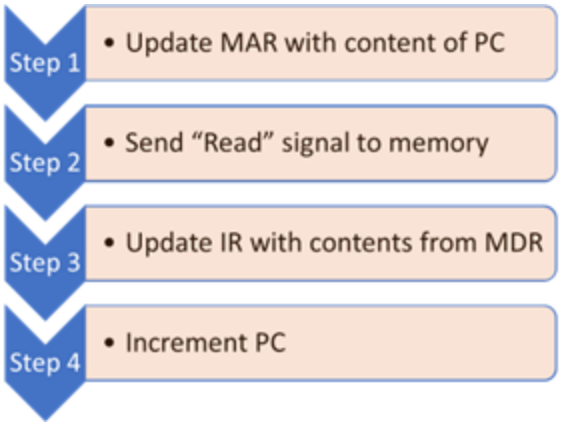
The instruction cycle has the following phases:

- **Fetch phase**: This phase obtains the next instruction to be processed from memory.
- **Decode phase**: We examine this instruction or, in other words, determine the action that is required to be taken. This helps set up the inputs to the ALU. It also provides us with the other operands needed to execute the operation.
- **Evaluate address phase**: If any of the operands are to be read from a memory location, the memory address of that location is computed in this phase.
- **The fetch operands phase** then reads the source operand from the computed memory address.
- The **execute phase** then performs the required operation, as identified by the opcode in the instruction.
- The **store result phase** stores the result of the operation at some destination.

Once an instruction is processed, this cycle is repeated for the next instruction, hence the term “instruction cycle.”

Instruction Cycle: Fetch Phase

The fetch phase gets the instruction to be processed from a memory location, whose address is stored in the program counter. This instruction is read from the memory location into the instruction register. Multiple steps are involved in the execution of the fetch phase.



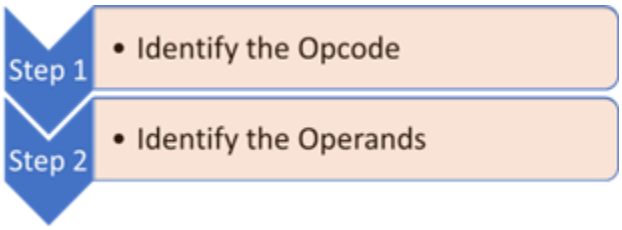
Each of these steps is carried out by the control unit. The first step is to copy the address contained in the program counter into the memory address register. This operation takes one machine cycle (or clock cycle).

Step 2 is to send the read signal to the memory. As per the von Neumann model, this read signal takes the content of the memory location that is addressed by the memory address register and copies the content to the memory data register. As memory access is generally slow, it may take more than one clock cycle to perform this step.

Once the memory data register has the result of the memory read operation, the next step is to read this instruction from the MDR into the instruction register. This is step 3 of the fetch phase.

Lastly, in step 4, the program counter is incremented by one. Incrementing the program counter by one ensures the next instruction in the sequence is fetched from memory. However, if the currently fetched instruction happens to be a control-type instruction, the program counter may be changed differently during the other phases of the instruction cycle.

Instruction Cycle: Decode Phase



The decode phase is the second phase of the instruction cycle. In this phase, we examine the instruction to decide what is to be done by the instruction. The decode phase can also be seen as a two-step process.

The first step is to identify the opcode in the instruction, that is, identify bits 12 to 15 of the instruction. In the LC-3, therefore, a 4-to-16 decoder is used to identify which one of the 16 possible opcodes is carried by this instruction. Feeding the 4 bits (bits 12 to 15) to such a decoder will result in one of the control lines being asserted, which will correspond to our opcode.

Once the correct decoder output is asserted, the second step is to examine the other bits, that is, the other 12 bits of the LC-3 instruction, to identify the operands. The format for each instruction differs, as we saw with the LC-3 instruction examples. So, the number and placement of the operands in the other 12 bits must be done based on the opcode.

Instruction Cycle: Evaluate Address Phase

Evaluate address is the third phase of the instruction cycle. This phase is only needed when any of the operands are to be read or stored in a memory location. If so, the address of the memory

location is evaluated in this phase. In the LC-3 Load (LD) instruction example we saw earlier, the PC+Offset addressing mode was used to specify the memory location to be read. The exact memory address to be read was calculated by sign extending the 2's complement integer stored in the instruction and adding it with the value in the program counter. This calculation is what the evaluate address phase performs.

Notice that many instructions may not need to either read or store any content in memory. The other LC-3 example we covered of the ADD instruction specified only the use of registers for source and destination. No memory access was needed for the ADD instruction example. Accordingly, such instructions do not have the evaluate address phase in their instruction cycle.

Instruction Cycle: Fetch Operands Phase

In the fetch operands phase, we are fetching the source operands required for performing the operation desired by the instruction. These operands may be fetched from the memory address evaluated in the evaluate address phase. Or they may be read from a register specified in the instruction. The source operands may even be specified in the instruction directly in the form of a value.

In case the source operand needs to be fetched from memory, this is again a multi-step procedure that involves writing the address into the memory address register, setting the read signal to the memory, and then finding the operand in the memory data register.

Instruction Cycle: Execute Phase

The fifth phase in the instruction cycle is the execute phase. The execution will depend on the instruction opcode itself. For the example of the LC-3 ADD instruction, this phase will involve performing the addition operation using the ALU. For the LC-3 JMP instruction example, the execute phase results in the program counter value getting overwritten from what was updated in the fetch phase. Lastly, for the LC-3 Load (LD) instruction, nothing is done in the execute phase. In general, data movement type instructions do not have any execute phase, as there is nothing to be executed.

Instruction Cycle: Store Result Phase

The last phase in the instruction cycle is the store result phase. In this phase, the result of the execute phase is written to the destination, whether memory or register. Suppose the result is to be written to memory. In that case, it again becomes a multi-step process, with the memory location address written to the MAR, the result written to the MDR, and then the write-signal enabled to memory.

This is the entire instruction cycle!